

JavaConfiguration Approach

|- class which have the source code

```
class A {
    B b;

    public void setB(B b) {
        this.b=b;
    }
}
```

```
class B {
}
```

xml approach

```
<beans>
  <bean id="a" class="package.A">
    </bean>
  <bean id="b" class="package.B">
    </bean>
</beans>
```

Annotation Approach

```
@component
class A {
    B b;

    @Autowired
    public void setB(B b) {
        this.b=b;
    }
}
```

```
@component
class B {
}
```

```
ApplicationContext context=
    new AnnotationConfigApplicationContext()
```

//No SOURCE CODE

```
class Rocket {
    Launcher launcher;

    @Autowired
    public void setLauncher(Launcher launcher) {
        this.launcher=launcher;
    }

    public void ignite() {
        launcher.powerSupplied();
        System.out.println("Rocket ignited");
    }
}
```

```
class Launcher {
    public void powerSupplied() {
        System.out.println("Fuel & setup for rocket are all set");
    }
}
```

```
@Configuration
class JavaConfig {
    @Bean (autowire=Autowire.byName)
    public Rocket rocket() {
        return new Rocket();
    }
}
```

```
@Bean
public Launcher launcher() {
    return new Launcher();
}
```

//There is one way we can inject which is manual way
 Rocket rocket=new Rocket();
 rocket.setLauncher("launcher");
 manually we have to write all the setter before!

Solution : If IOC container will manage it then all will be fine

```
<bean id="rocket" class="packageName.Rocket" autowire="byName">
  <property name="launcher" ref="launcher">
    </bean>
  </bean>
<bean id="launcher" class="packageName.Launcher">
  </bean>
```

let's see the example with autowired with java configuration approach

```
package com.day27;

public class Rocket {
    Launcher launcher;
    //Repair Center

    public void setLauncher(Launcher launcher) {
        this.launcher = launcher;
    }

    public void ignite() {
        launcher.powerSupplied();
        System.out.println("Rocket is set to go !!!");
    }
}
```

```
package com.day27;

public class Launcher {

    public void powerSupplied() {
        System.out.println("Rocket fuel & all setup done!!");
    }
}
```

```
package com.day27;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.AnnotationConfig;
import org.springframework.context.annotation.AnnotationConfig;

@Configuration
public class AutowireConfig {

    @Bean(autowire=Autowire.BY_TYPE)
    public Rocket rocket() {
        return new Rocket();
    }

    @Bean
    @Primary
    public Launcher launcher1() {
        return new Launcher();
    }

    @Bean
    public Launcher launcher2() {
        return new Launcher();
    }

    // Repair center bean definition 2
}
```

For working with this program we have to switch to the older version of spring core and context

```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-core</artifactId>
<version>5.3.39</version>
</dependency>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>5.3.39</version>
</dependency>
```

```
package com.day27;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class LaunchCenter {
    public static void main(String[] args) {

        ApplicationContext context=new
            AnnotationConfigApplicationContext(AutowireConfig.class);

        Rocket bean = context.getBean("rocket",Rocket.class);
        bean.ignite();
    }
}
```

let's see the example on
 |- @Bean, @DependsOn, @Scope, @Lazy
 with java configuration approach

```
package com.day27;

import org.springframework.stereotype.Component;

public class Student {

    public Student() {
        System.out.println("Student joined ");
    }

    public void work() {
        System.out.println("Working");
    }
}
```

```
package com.day27;

public class University {
    public University() {
        System.out.println("Student Verification");
    }
}
```

```
package com.day27;

import java.util.Scanner;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
        System.out.println("Main Started");
        ApplicationContext context = new
            AnnotationConfigApplicationContext(JavaConfig.class);
        // Student bean = context.getBean("student",Student.class);
        // Student bean2 = context.getBean("student",Student.class);
        // System.out.println(bean==bean2);
    }
}
```

```
package com.day27;

/*
 * By default the bean definition is of what scope
 * |- singleton
 *
 * means if we are writing inside the xml file
 * <bean id="student" class="package.Student" scope="">
 * </bean>
 * this bean definition is by default singleton
 *
 * now we want multiple object
 */
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Lazy;
import org.springframework.context.annotation.Scope;

@Configuration
public class JavaConfig {

    @Bean
    // @Scope("prototype")
    // @Lazy
    @DependsOn("university")
    public Student student() {
        return new Student();
    }

    @Bean
    public University university() {
        return new University();
    }
}
```